To you who are reading, hello!
My name is Beatrice Occhiena and I am a student of Artificial Intelligence.

Since I study so many different subjects on my own, I tend to rely a lot on all the resources I find on the web 🌐. Whenever I do some research online to learn something new, I always feel very lucky, because I find so much free material carefully crafted by other people from all over the world. So I too decided to make my own small contribution ✨.

Over the months I have put a lot of care into the creation of these notes ✍🏻. And although I know that they may be incomplete and imperfect in some places, I would like to make them available to you — in the hope that they may help even one student in the exam preparation.
I firmly believe in collaboration and mutual support between students 👐🏻. Furthermore, in the future I would love to be involved in research and education, so this is a first opportunity for me to showcase my work.

That said, I wish you all the best for this exam! Take these words as a small gesture of encouragement. Even though we don't know each other, I still want to cheer you on! Do your best and don't forget to have as much fun as possible studying this subject 💫.

Contact me!

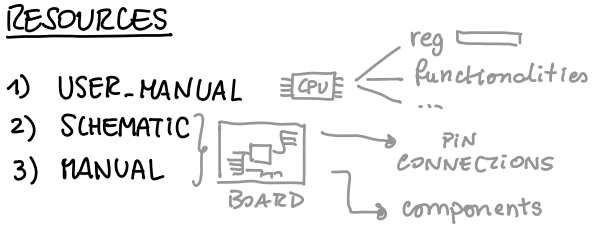
beatrice.occhiena

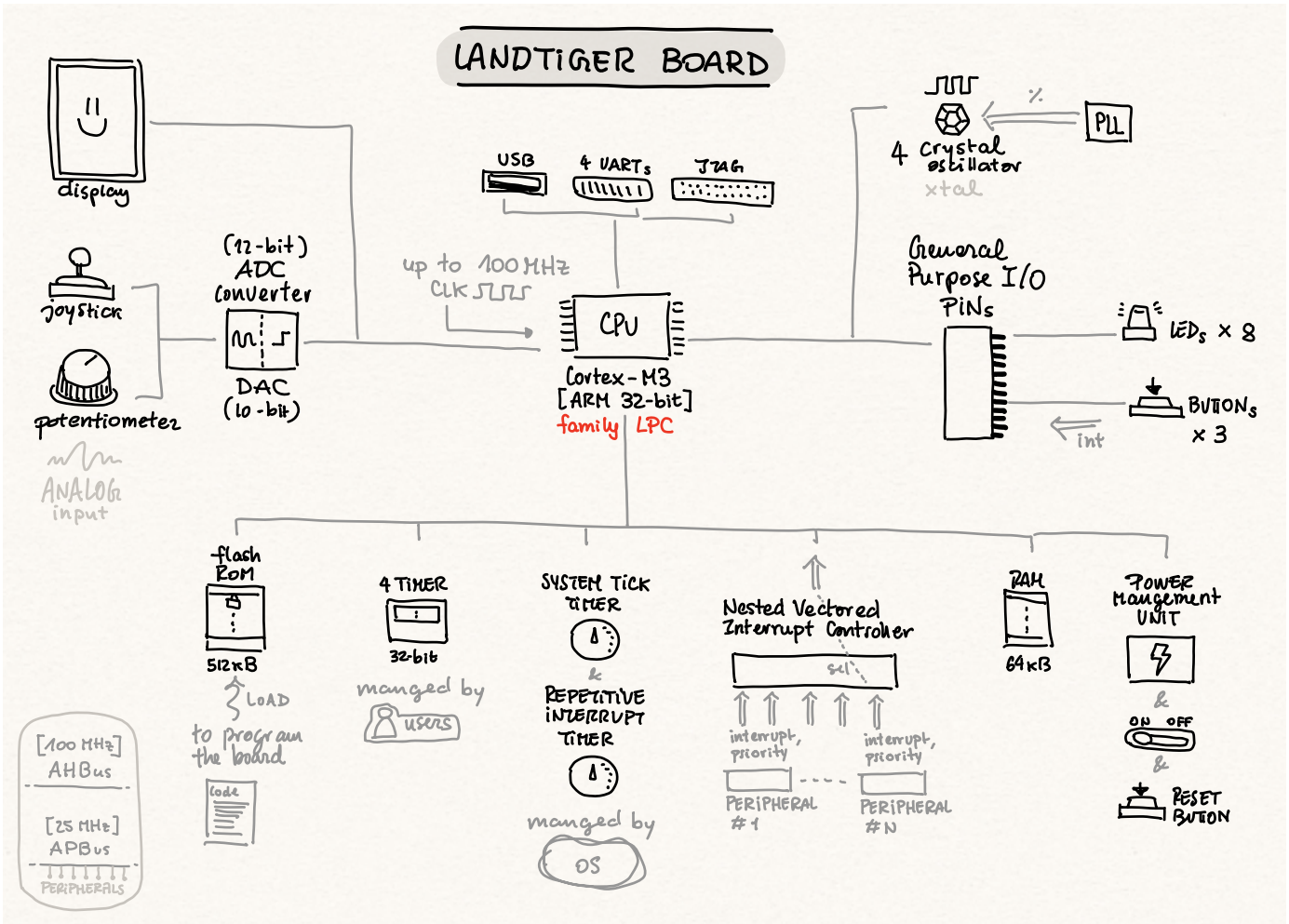
beatrice.occhiena@live.it

• PART 1: LandTiger Baseboard

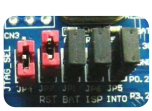
RESOURCES

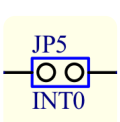
1) USER_MANUAL
2) SCHEMATIC
3) MANUAL





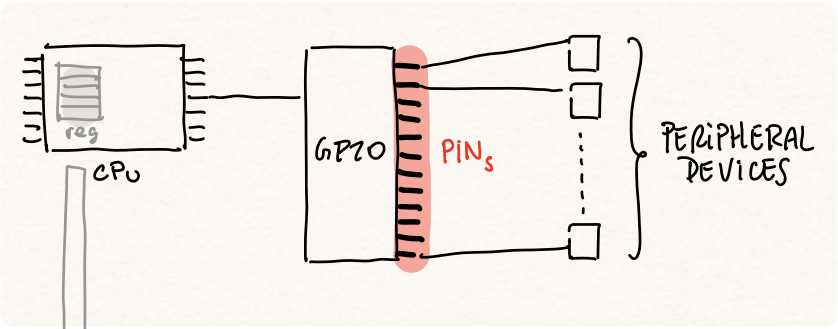
JUMPERS





short lenght of conductor
⇒ CLOSE - OPEN circuits on el. boards.
⇒ EN-DISABLE functions

GPIO = 'General Purpose Input - Output'

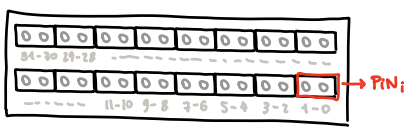


PINs ORGANIZED IN 5 GROUPS

P0.0 → group 0, pin n°0

POWER always ON

LPC_PINCON → PINSEL X

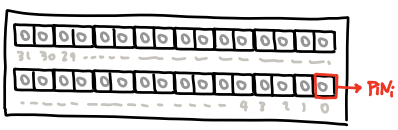


0 = P0.0 ... P0.15
1 = P0.16 ... P0.31
2 = P1.0 ... P1.15
3 = P1.16 ... P1.31
4 = P2.0 ... P2.15
5 = P2.16 ... P2.31

4 OPERATING MODES :

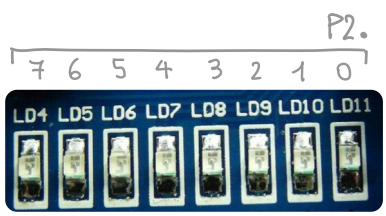
GPIO	00	→ LEDS
ADC1	01	→ BUTTONS (EINT_)
ADC2	10	
ADC3	11	

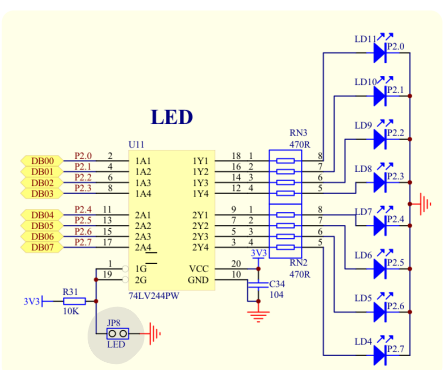
LPC_GPIOn → FIODIR (n = group n)



DIRECTIONED AS input 0 → BUTTONS (e.g.)
OR output 1 → LEDS

LEDS



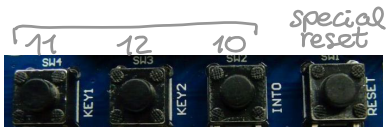


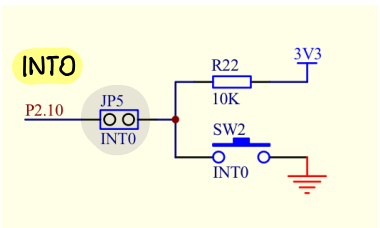
- Used in mode GPIO - output

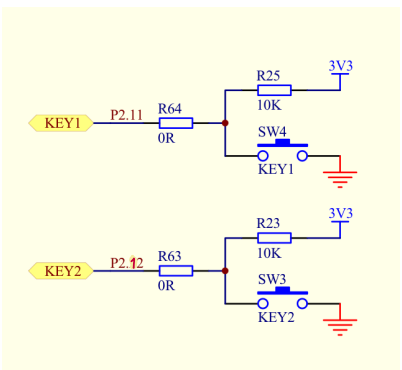
Functions

LED_On(n° led);
_Off(n° led);
_Out(value);

BUTTONS







P2.10 / P2.11 / P2.12

- Used in mode EINT - input 'External INTerrupt'

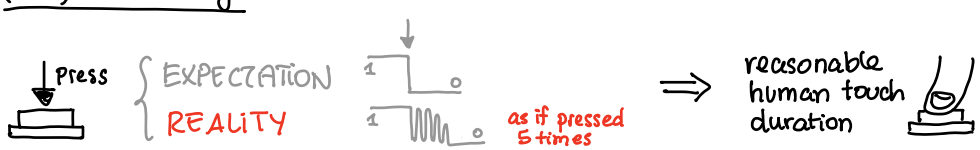
⇓

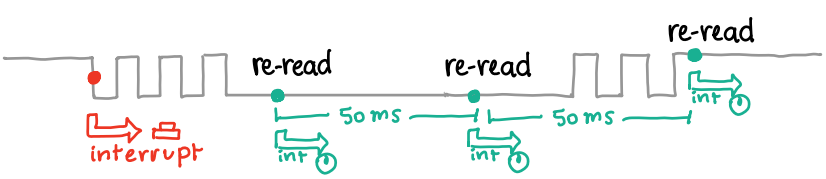
- They send an IRQ 'Interrupt Request' to the NVIC

Functions

```
EINTX_IRQHandler(void){
    // when the INTERRUPT is handled
    ...   (my code about what the button controls)
    LPC_SC -> EXTINT &= (1 << X);
    // ↑ To clear the pending interrupt
}
```

(DE)-Bouncing





- enables a RIT timer of 50 ms
- disables the button's INT
- sets button's pin to GPIO

RIT

PRESSED

read the PIN in mode GPIO

PIN = 0

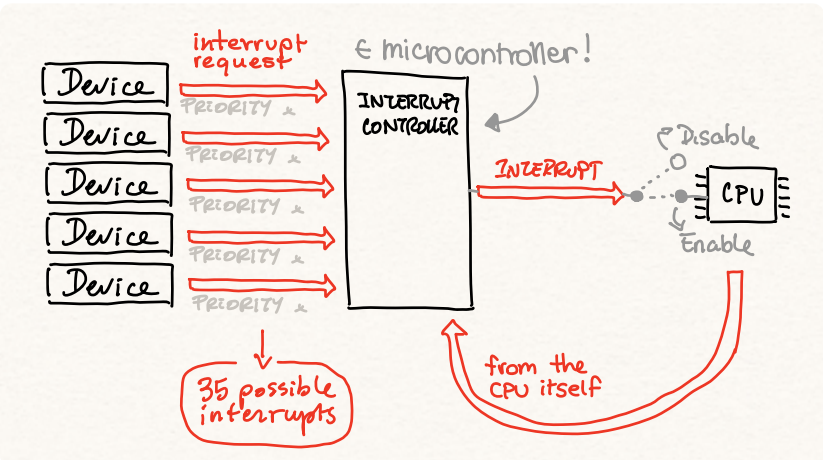
- reset RIT
- A if down=1: perform action
- counter
- B if down > 1: nothing

RELEASED

PIN = 1

- C disables and resets RIT
- C enable the button's INT

NVIC - 'Nested Vector Interrupt Controller'



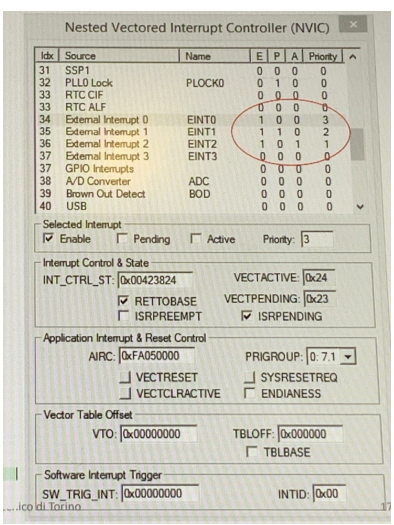
€ microcontroller!

interrupt

* ERROR / ALARM conditions
  - LOW POWER !
* I-O DEVICE synchro
  - PRESSED BUTON
* PERIODIC
  - timer TIME-OUT
* DATA ACQUISATION
  - ADC ready
* CPU EVENTS

Functions

```
NVIC_EnableIRQ ( EINTX_IRQn );
    DisableIRQ
NVIC_SetPriority ( EINTX_IRQn, 1 );
```



Priority

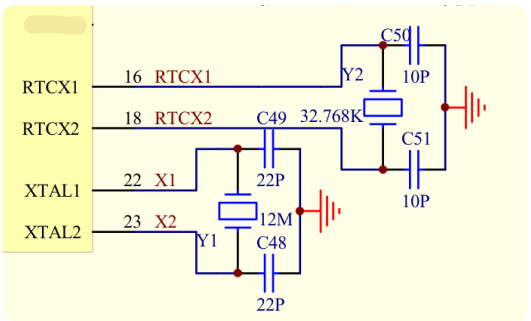
-2
-1
0
1
2
3

E Enabled (1) - DiS (0)

P Pending

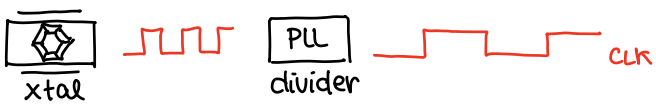
A In execution

CLOCK / OSCILLATORS



CLK (1 MHz ↔ 25 MHz)

1) System clock (xtal 12 MHz)
2) Real Time clock RTC
3) Ethernet clock (4 MHz)
4) Debugger interface clock



SystemInit(); ← configuration in here with register LPC_SC → ...

POWER CONTROL

OB: reduce dynamic power consumption

STOP CLK ⎍⎍ UNTIL INTERRUPT ⊕ PERIPHERAL POWER CONTROL

	CPU	BOARD	
	SLEEP	DEEP SLEEP	
new OFF	POWER DOWN	DEEP POWER DOWN	latency time to wake-up

to TURN OFF CLK ⎍⎍ for individual peripherals
↳ if not in use

reg: SC — "System Control"

31 0

reg: PCON

31 1 0

- 00 = (deep) sleep
- 01 = power down
- reserved
- 11 = deep power down

set after the execution of
_ASM("wfi");
assembly in-line command
"Wait For Interrupt"

reg: PCONP

31 0

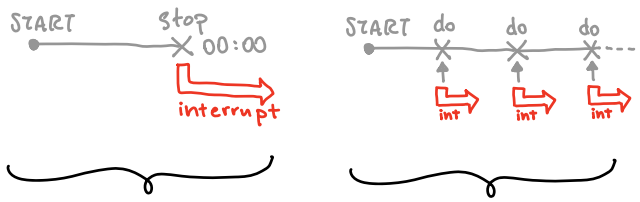
each BIT controls 1 peripheral

Some PERIPHERALS are SHUT DOWN (0) by DEFAULT ⇒ I must activate them! (1)

The SIMULATOR cannot see this problem, but the BOARD crashes!

TIMERS

- Wait X_seconds
- perform regular operations



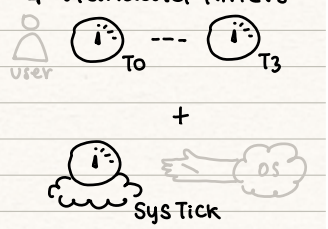
while counting	while handling the int
reduced power	full power

CLK

$count = time[s] \cdot freq_{CLK}\left[\frac{1}{s}\right]$

CORE DEVICES at 100 MHz	PERIPHERALS at 25 MHz
0B : wait 10ms	0B : wait 10s
COUNT = 0x000F4240	COUNT = 0x0EE6B280

4 standard timers



+

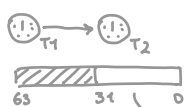
RIT = "Repetitive Interrupt Timer"

+

PWM — generates a wave function (DUTY CYCLE)

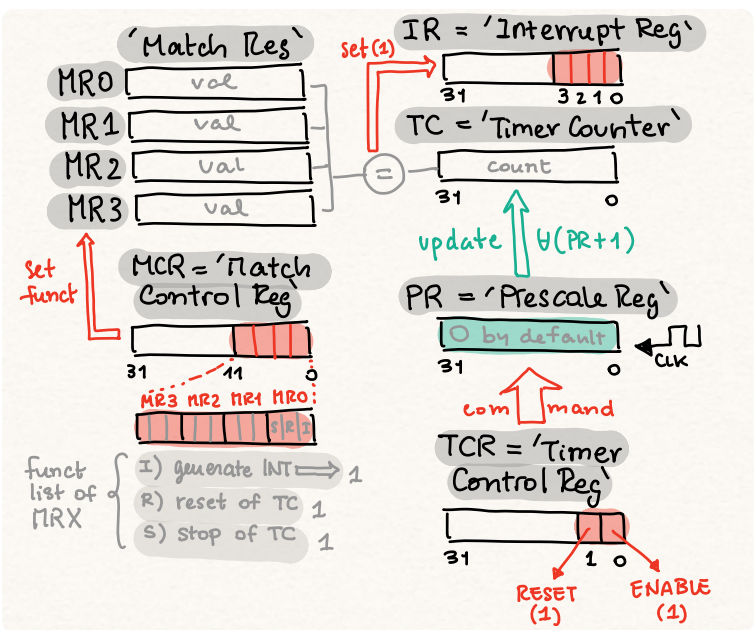
TIME LIMITS

- HW - CASCADE
- HW - PRESCALER
- SW - conting nº interrupts



$count = time[s] \cdot \frac{freq_{CLK}\left[\frac{1}{s}\right]}{(PR+1)}$

Registers



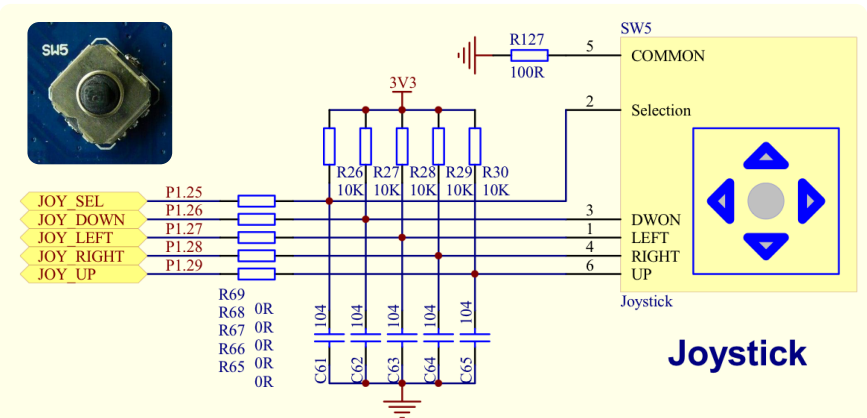
Functions

prescaler, MRn, SRIfunc in HEXADECIMAL

- init_timer(num, count);
- enable_timer(num);
- disable
- reset

- TIMERX_IRQHandler(){
  - ... code for handling
  - LPC_TIMX → IR = 1; clear interrupt flag
  - }

JOYSTICK



- NOT INTERRUPT
  ⇒ polling with RIT
  init & enable in sample.c
- HW MANAGEMENT of DEBOUNCING

GPIO pins "OUTPUT"

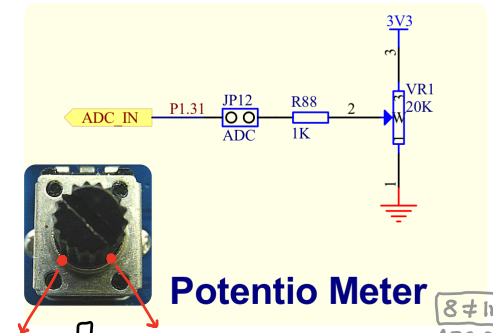
P1.25	select ≡ PRESSED
P1.26	down
P1.27	left
P1.28	right
P1.29	up

can be activated simultaneously (e.g. UP-LEFT)

PROBING in RIT_IRQHandler(){

```
if((LPC_GPIO1 -> FIOPIN & (1<<25)) == 0){
    // if JOYSTICK PRESSED
    ---
}
```

Potentiometer (input for ADC)



0V min

3.3V max

voltages between [0,3.3]V

8 ≠ Inputs

AD0.0

AD0.5

AD0.7

in ADC out

2^{12} levels

value on 12-bit

11 0

⚠ Non linear inputs!

1.232V 1.241V 1.237V...
keeping the same pos

fixed

conversion rate 200 kHz

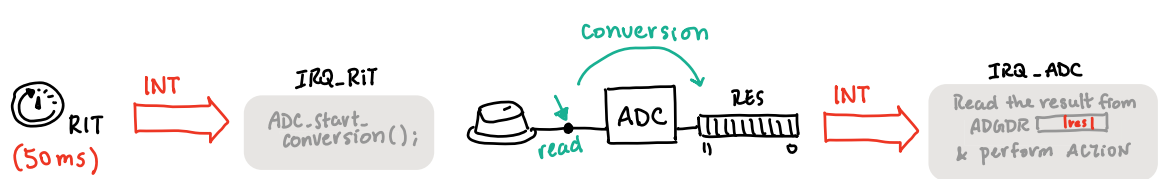
ADC_init(){
* input pin selection (1 << 5) ≡ AD0.5
* clock selection (4 << 8) ≡ 5 MHz
* enable ADC (1 << 21)
}

ADC_start_conversion(){
* start converting (1 << 24)
}

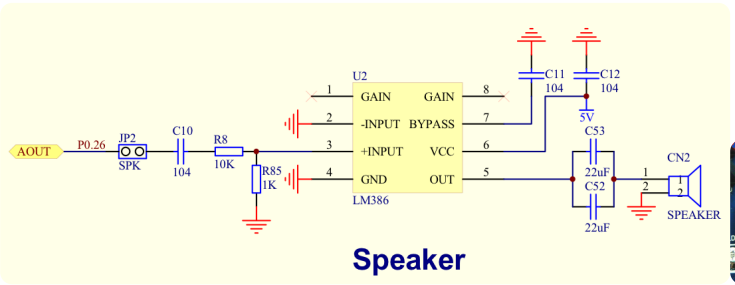
ADCR — Selects the operating mode (31 … 0)

ADGDR — 12-bit RESULT (31 … 15 … 4 … 0)
0x000 → 0V
⋮
0xFFF → 3.3V
Contains the res of last conversion from All inputs

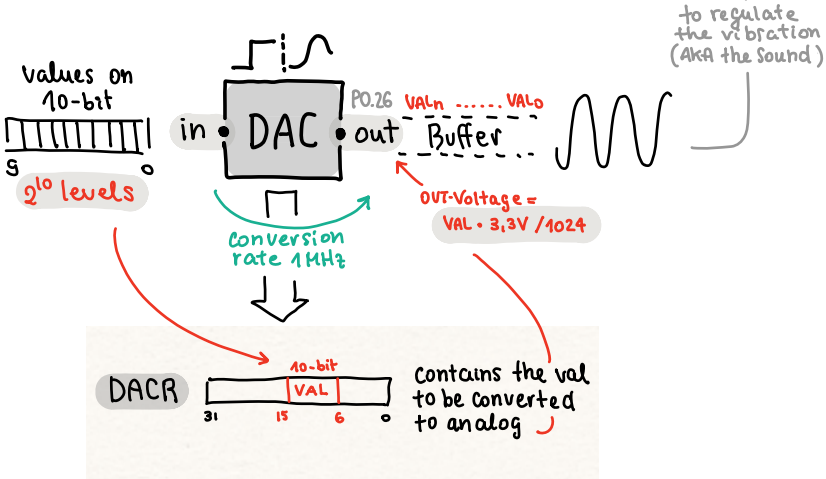
ADODRX — 12-bit RESULT (31 … 15 … 4 … 0)
0x000 → 0V
⋮
0xFFF → 3.3V
Contains the res of last conversion from ✕ input

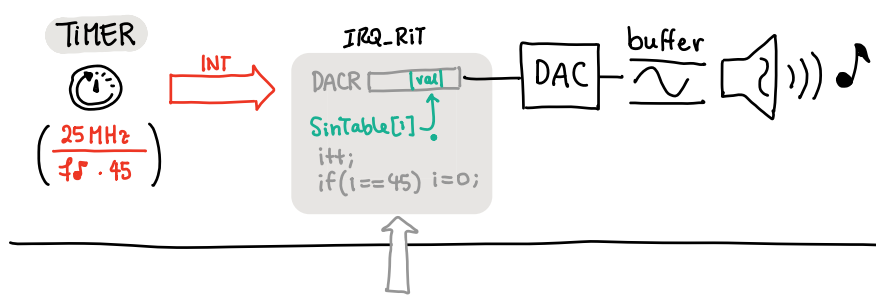


Speakers (output for DAC)

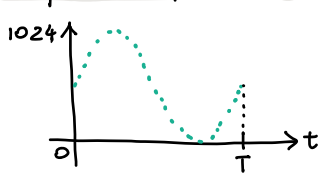


to regulate the vibration (AKA the sound)





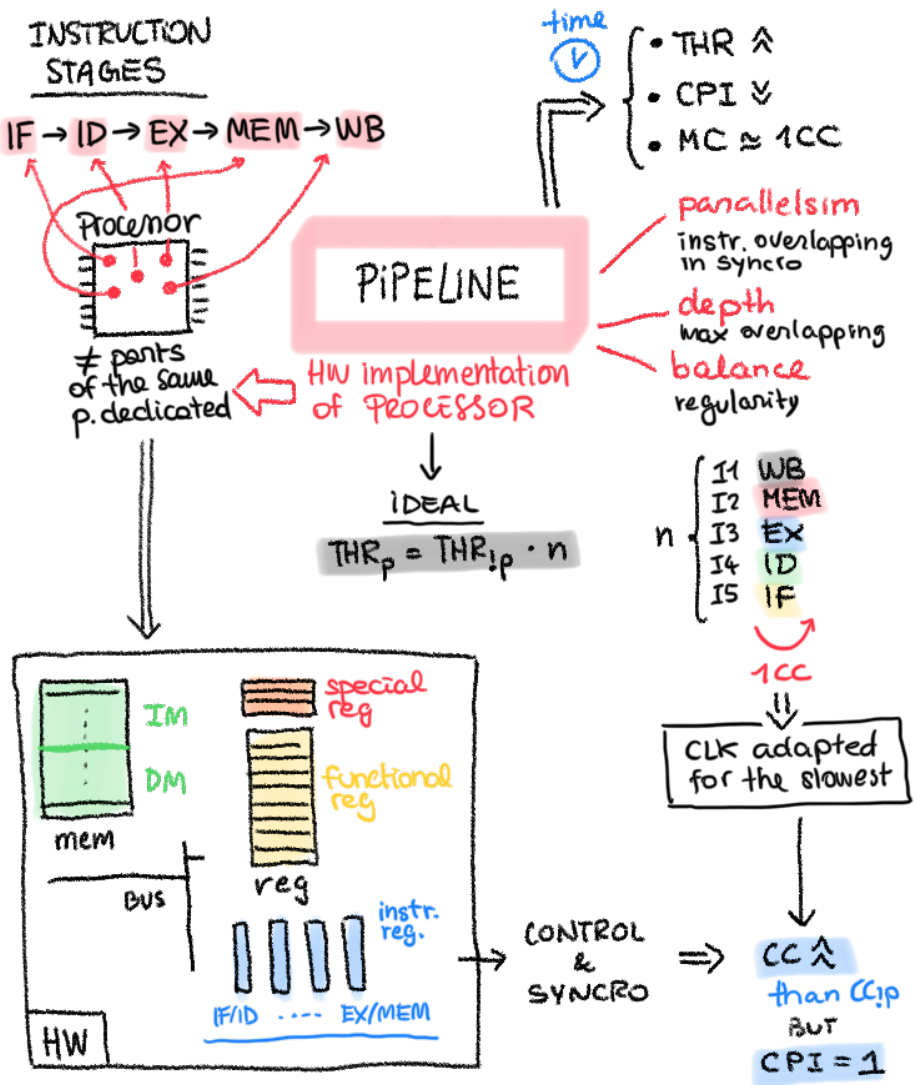
Sampled sin function

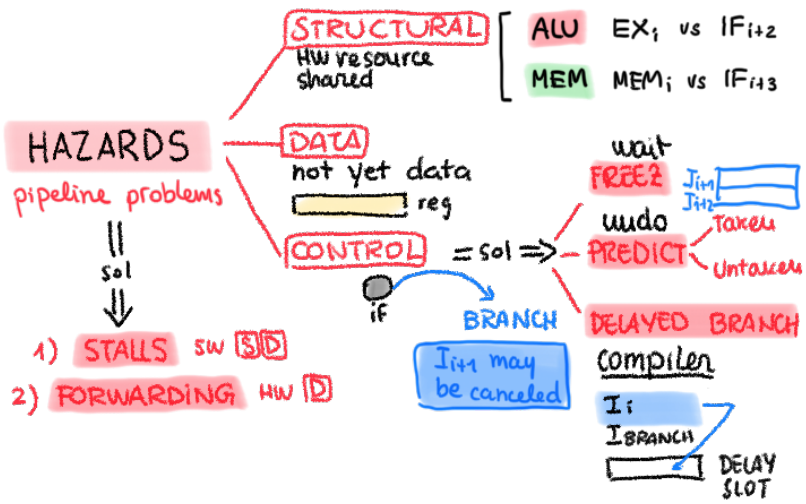


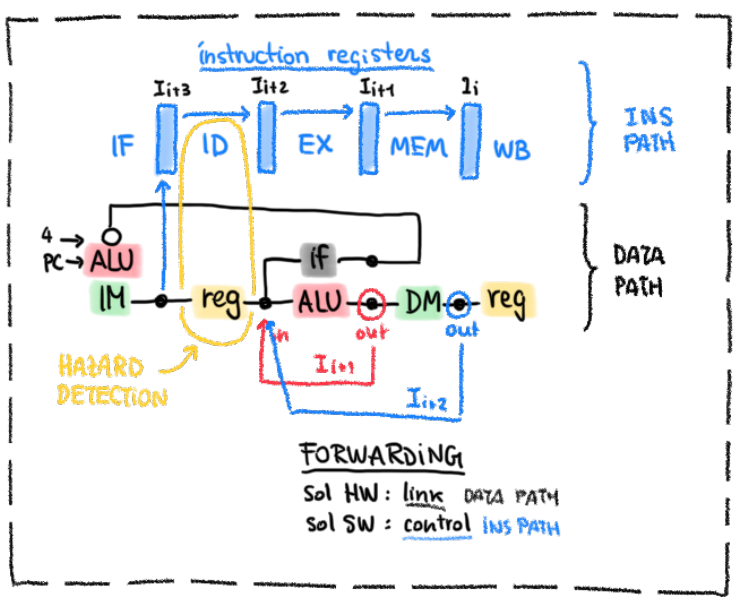
* Amplitude = VOLUME
* Frequency = NOTE (A = 440 Hz)

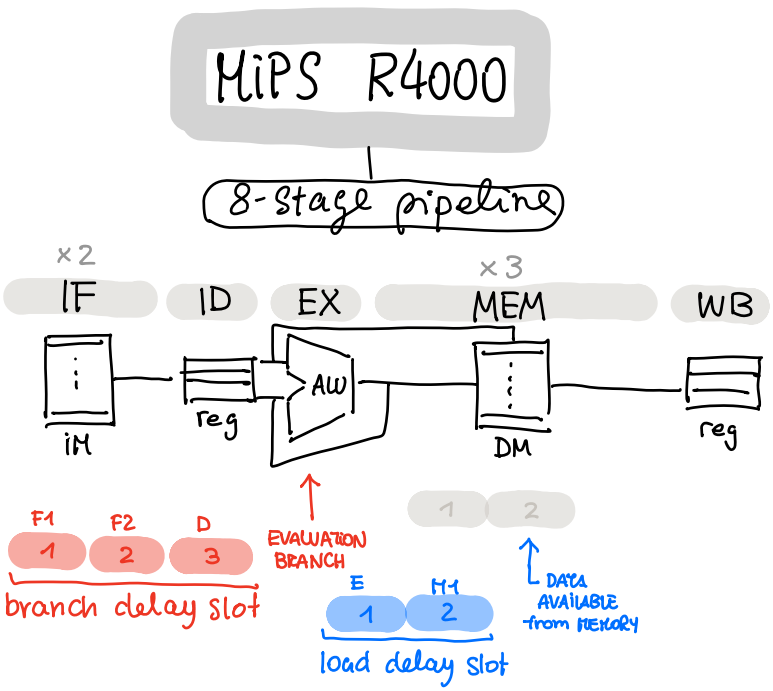
uint16_t SinTable[45] =
{410, 467, 523, ---, 0, 6, 20, ---, 353}

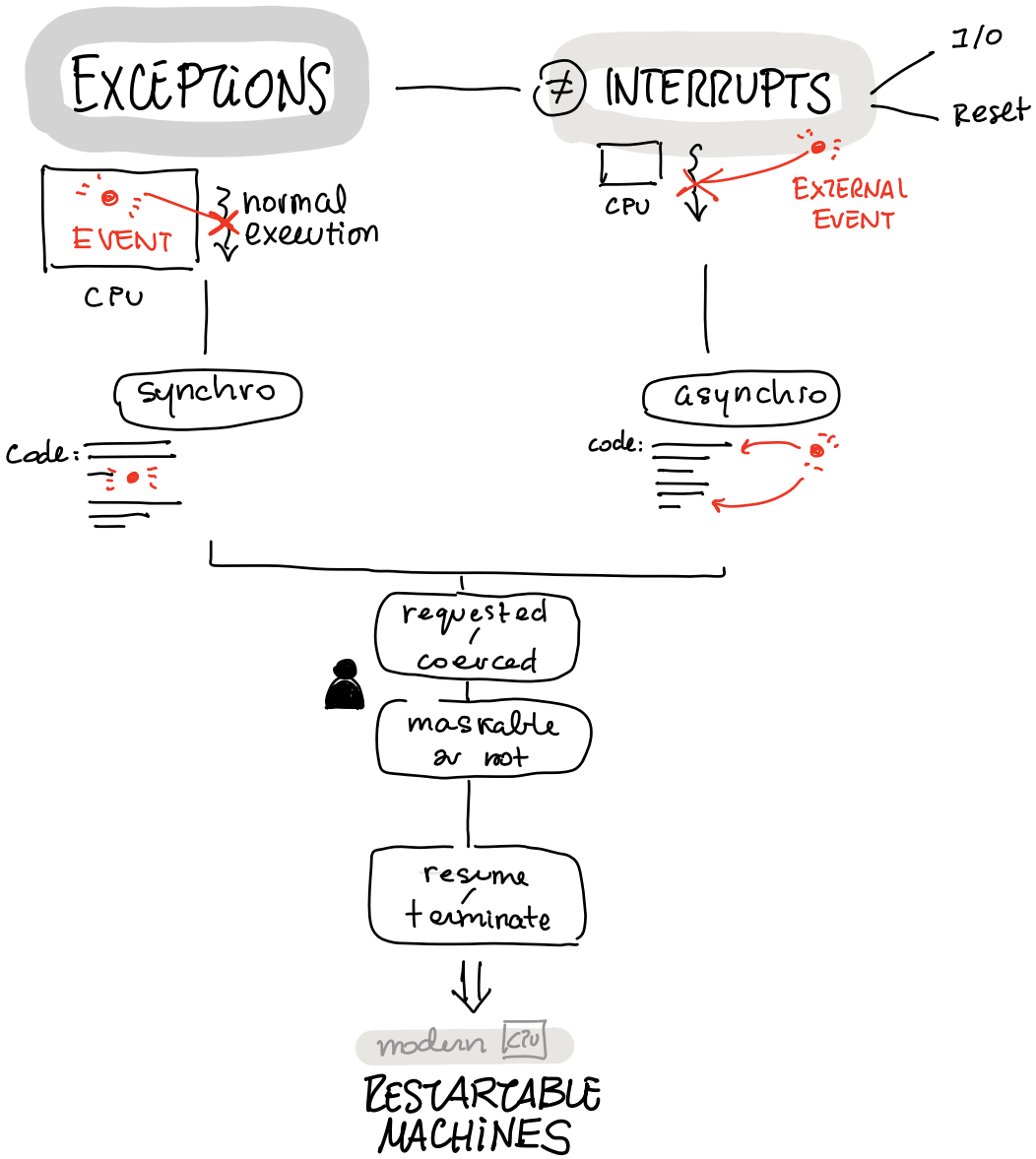
• PART 2 : A bit of theory

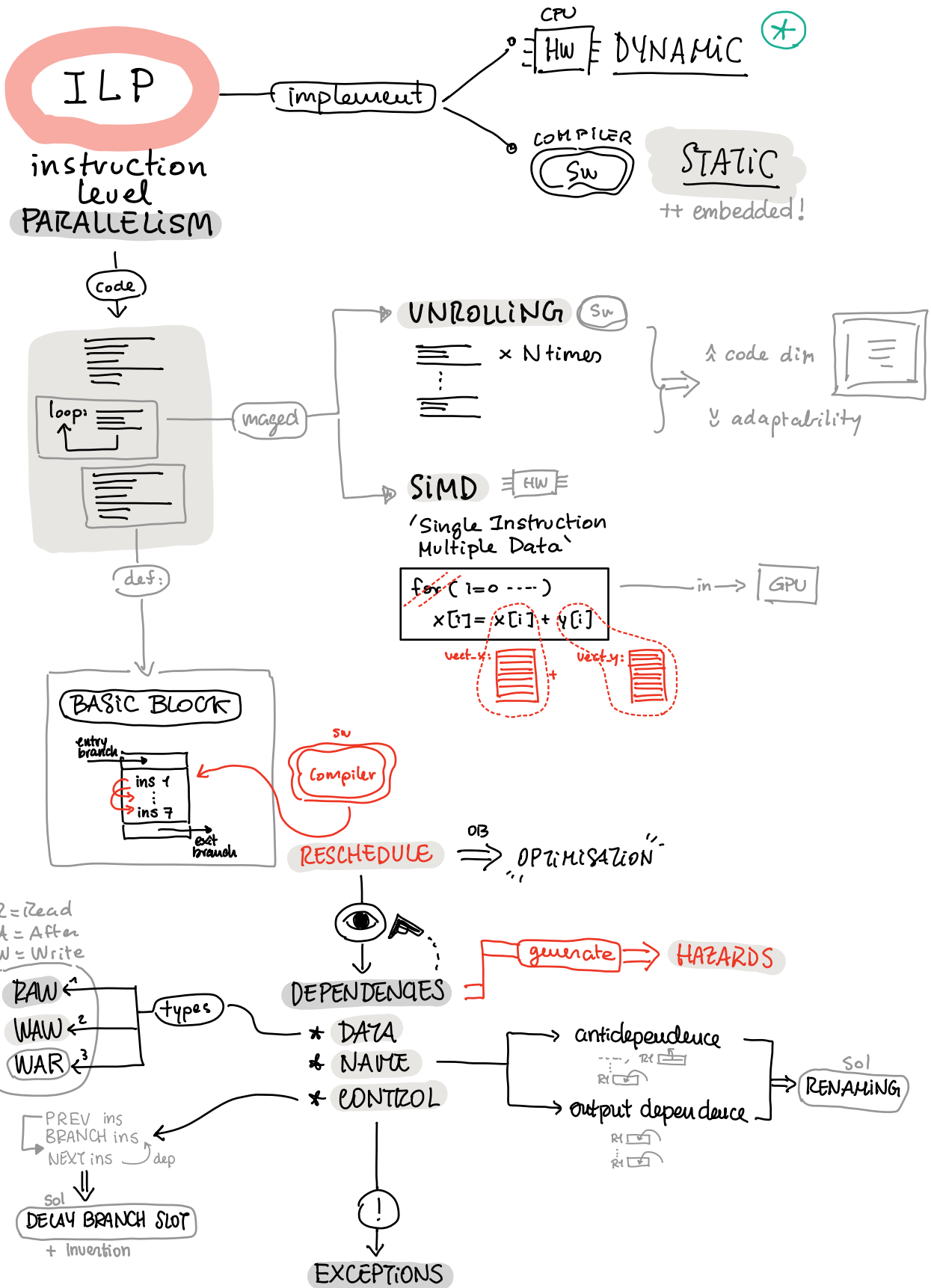


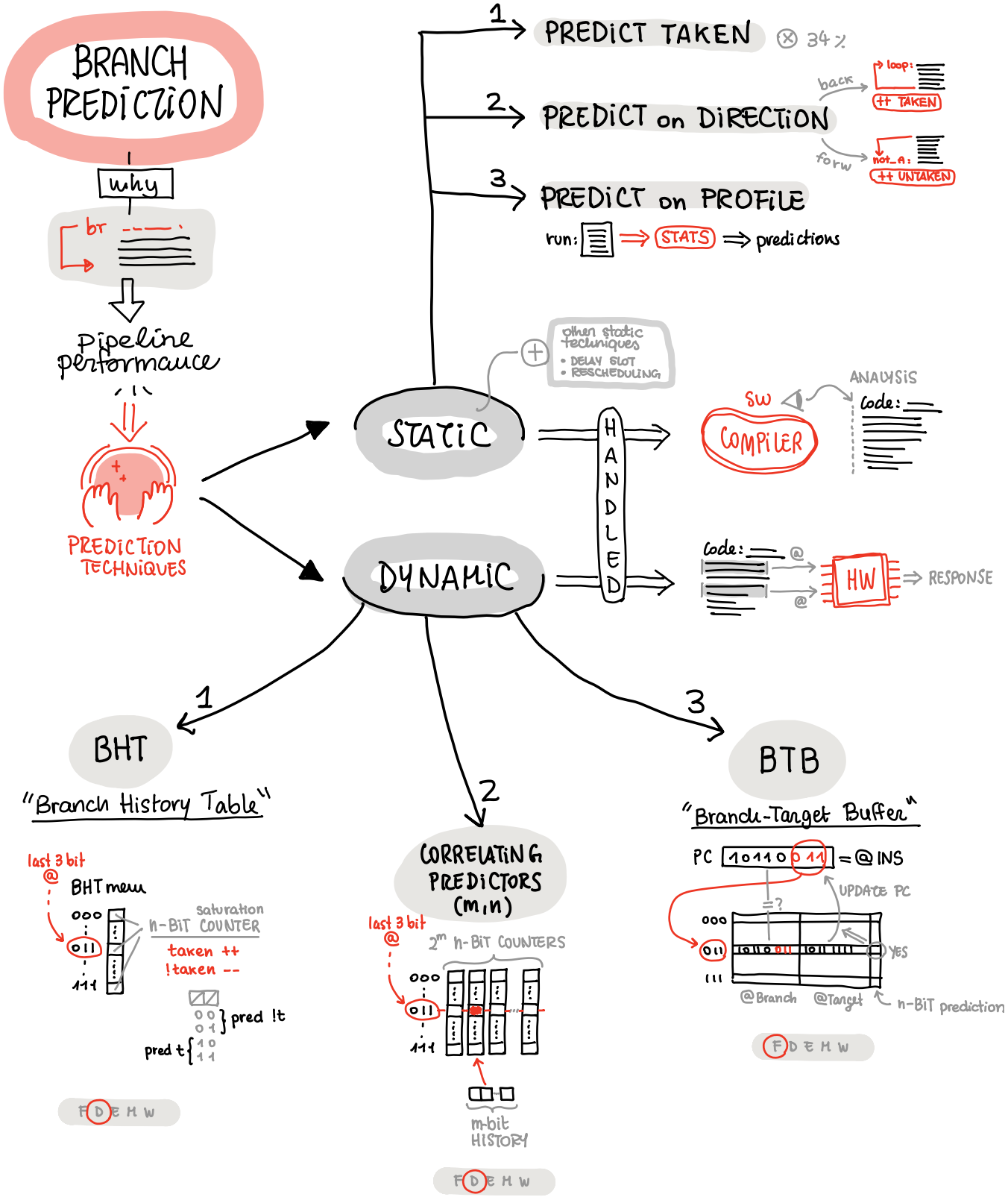

HAZARDS
pipeline problems
sol
1) STALLS SW SD
2) FORWARDING HW D
STRUCTURAL
HW resource shared
ALU EX_i vs IF_i+2
MEM MEM_i vs IF_i+3
DATA
not yet data
reg
CONTROL
= sol
if
BRANCH
I_i+1 may be canceled
wait
FREEZ
I_i+1
I_i+2
undo
PREDICT
Taken
Untaken
DELAYED BRANCH
compiler
I_i
I_BRANCH
DELAY SLOT


instruction registers
I_i+3
I_i+2
I_i+1
I_i
IF
ID
EX
MEM
WB
INS PATH
4
PC
ALU
IM
reg
if
ALU
DM
reg
in
out
out
DATA PATH
HAZARD DETECTION
I_i+1
I_i+2
FORWARDING
Sol HW : link DATA PATH
Sol SW : control INS PATH


MiPS R4000
8-stage pipeline
×2
×3
IF
ID
EX
MEM
WB
IM
reg
ALU
DM
reg
F1
F2
D
1
2
3
branch delay slot
EVALUATION BRANCH
E
M1
1
2
load delay slot
DATA AVAILABLE from MEMORY


EXCEPTIONS
≠
INTERRUPTS
I/O
Reset
EVENT
normal execution
CPU
CPU
EXTERNAL EVENT
synchro
asynchro
code:
code:
requested / coerced
maskable or not
resume + eliminate
modern CPU
RESTARTABLE MACHINES


ILP
instruction level PARALLELISM
implement
CPU
HW
DYNAMIC
COMPILER
SW
STATIC
++ embedded!
code
loop:
maged
UNROLLING
SW
x N times
↑ code dim
↓ adaptability
SIMD
HW
'Single Instruction Multiple Data'
for (i=0 ----)
x[i] = x[i] + y[i]
in → GPU
vect_x:
vect_y:
def:
BASIC BLOCK
entry branch
ins 1
ins 7
exit branch
SW
Compiler
RESCHEDULE
OB
OPTIMISATION
R = Read
A = After
W = Write
RAW
WAW
WAR
types
DEPENDENCES
generate
HAZARDS
* DATA
* NAME
* CONTROL
antidependence
output dependence
Sol
RENAMING
PREV ins
BRANCH ins
NEXT ins
dep
Sol
DELAY BRANCH SLOT
+ invention
EXCEPTIONS


BRANCH PREDICTION
why
br
Pipeline performance
PREDICTION TECHNIQUES
1
PREDICT TAKEN
⊗ 34 %
2
PREDICT on DIRECTION
back
loop:
++ TAKEN
forw
not_a:
++ UNTAKEN
3
PREDICT on PROFILE
run: ⇒ STATS ⇒ predictions
STATIC
other static techniques
• DELAY SLOT
• RESCHEDULING
HANDLED
SW
COMPILER
ANALYSIS
code:
DYNAMIC
code:
@
@
HW
⇒ RESPONSE
1
BHT
"Branch History Table"
last 3 bit
@
BHT mem
000
011
111
saturation
n-BIT COUNTER
taken ++
!taken --
00
01
} pred !t
pred t { 10
11
F D E M W
2
CORRELATING PREDICTORS (m,n)
last 3 bit
@
2^m n-BIT COUNTERS
000
011
111
m-bit HISTORY
F D E M W
3
BTB
"Branch-Target Buffer"
PC 10110 01 1 = @ INS
UPDATE PC
=?
000
011
111
YES
@Branch
@Target
n-BIT prediction
F D E M W

HW

DYNAMIC SCHEDULING

problem

DATA DEPENDENCY

div.d F0, F1, F2

add.d F3, F0, F1

Sub.d F6, F4, F5

↳ POINTLESS WAITING

sol

OB

OUT-OF-ORDER EXCUTION

⟹ ⚠ imprecise exceptions

TOMASULO'S APPROACH

keys

RESERVATION STATIONS

=

	Op	V_j	V_k	Q_j	Q_k	A	Busy
div1		val F1	val F2				yes
add1			val F1	div1			yes

INS waiting for execution

RENAMING

for val not yet available

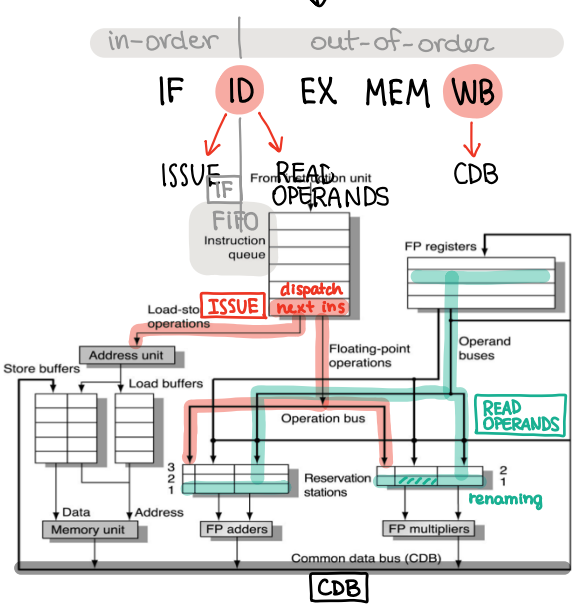
F0 → div1

INS will wait for the result of div1,

COMMON DATABUS

the result of a INS is available for every INS waiting at the same time

pipeline



extended for

SPECULATION

OB

EFFECTS of BRANCHES → PREDICT → EXECUTE ASAP !!!

but results not saved in definitive reg or mem@

introduce

ISSUE

RoB "ReOrder Buffer"

Implement

CIRCULAR BUFFER

INS type	DEST	VAL	READY
INS ...			
INS ...			
next INS			

INS original order

COMMIT

MISPREDICT BRANCH → clear

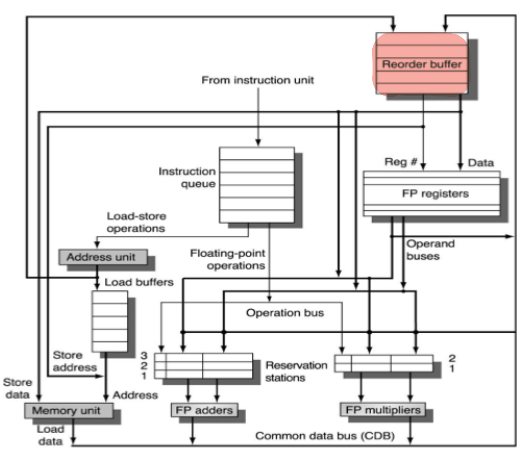
OTHERWISE → INS nomore SPECULATIVE → val written in register (val) / mem@ (val) of destination

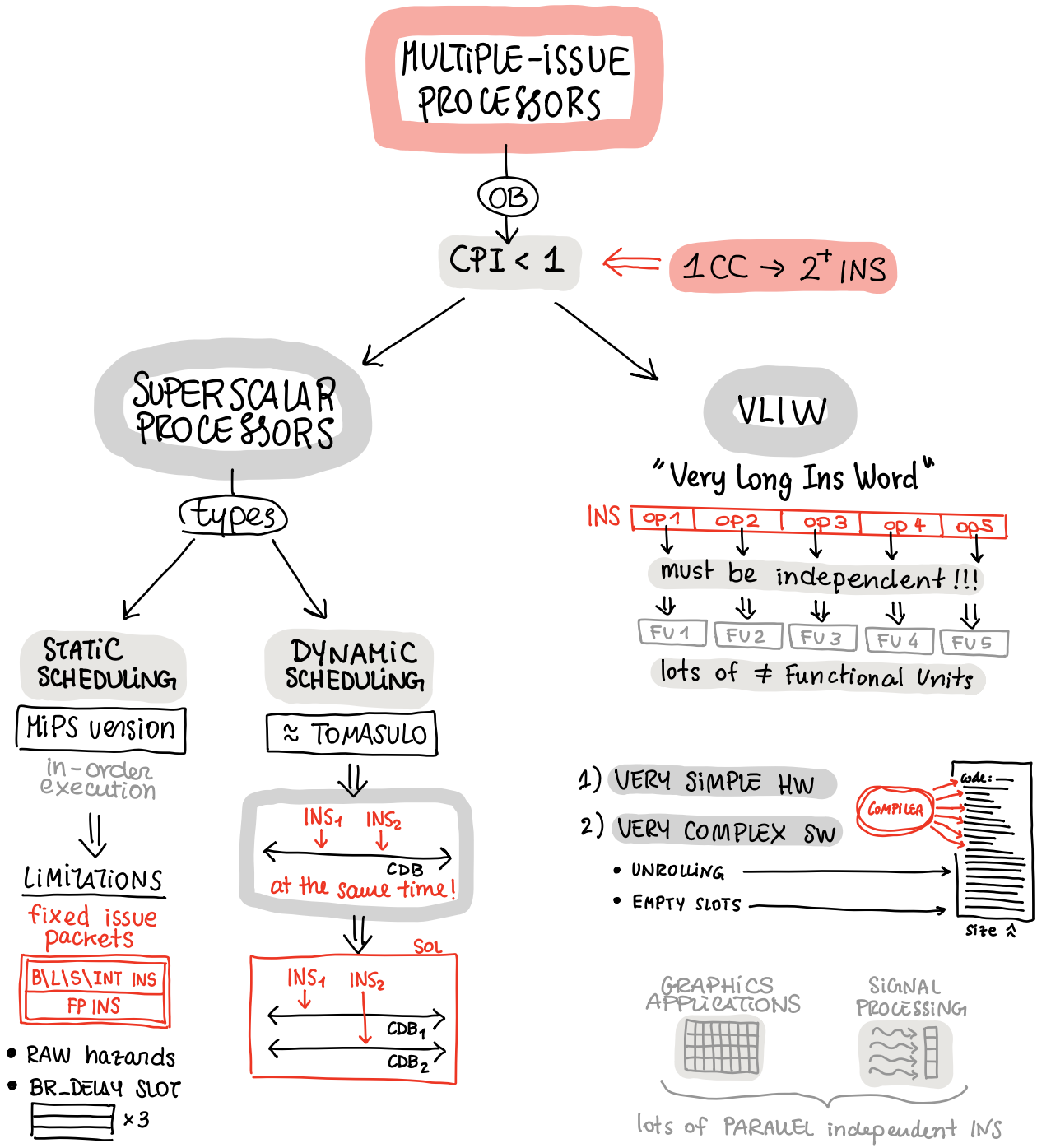
STORE INS write in mem only during commit

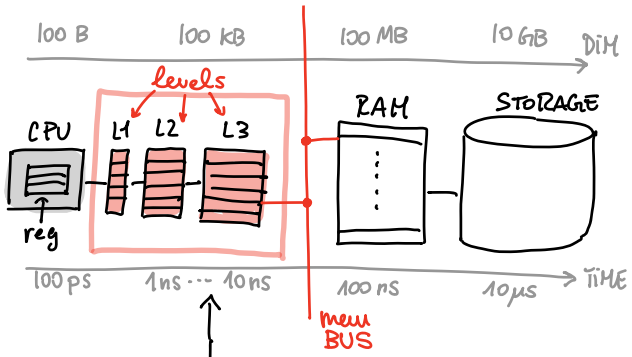
FULLY PRECISE EXCEPTION HANDLING

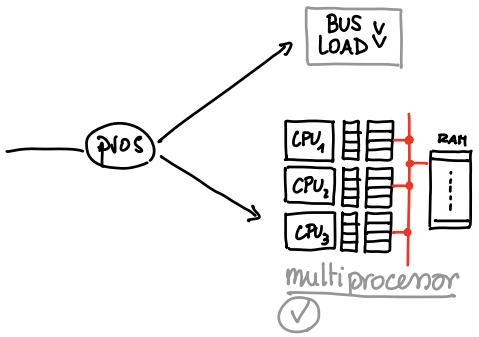
new pipeline

in-order	out-of-order				in-order
IF	ID	EX	MEM	WB	COMMIT
	ISSUE	READ OPERANDS		CDB	




MULTIPLE-ISSUE PROCESSORS
OB
CPI < 1
1 CC → 2+ INS
SUPERSCALAR PROCESSORS
types
STATIC SCHEDULING
MIPS version
in-order execution
LIMITATIONS
fixed issue packets
B\L\S\INT INS
FP INS
• RAW hazards
• BR_DELAY SLOT
×3
DYNAMIC SCHEDULING
≈ TOMASULO
INS1 INS2
CDB
at the same time!
SOL
INS1 INS2
CDB1
CDB2
VLIW
"Very Long Ins Word"
INS
op1 op2 op3 op4 op5
must be independent !!!
FU1 FU2 FU3 FU4 FU5
lots of ≠ Functional Units
1) VERY SIMPLE HW
2) VERY COMPLEX SW
• UNROLLING
• EMPTY SLOTS
COMPILER
code:
size
GRAPHICS APPLICATIONS
SIGNAL PROCESSING
lots of PARALEL independent INS



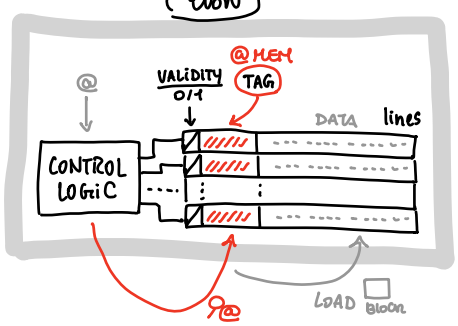


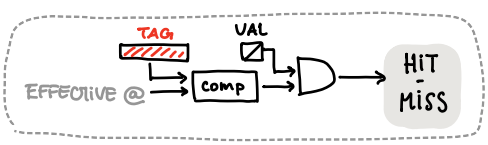
where

CACHE

def: Small high-speed memory

how





why

performance ↑

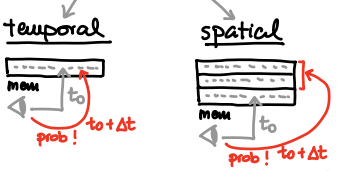
HIT RATIO

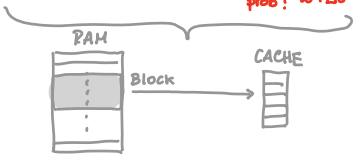
$t_{ave} = h \cdot C + (1-h) \cdot M$

CACHE acc time

MEM acc time

PRINCIPLE of LOCALITY

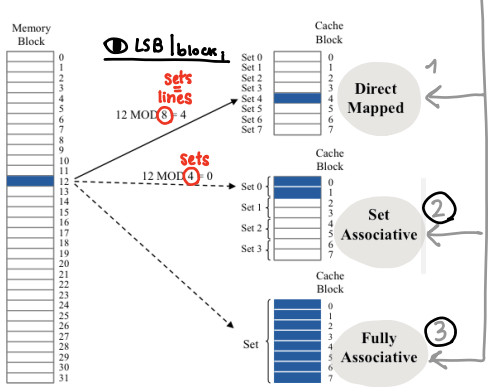




parameters

- size
- mapping
- replace algorithm
- mem update mod.

MAPPING efficient OB: @



IN which LINE ?

REPLACING ALG.

LRU ++ = least recently used

HISTORY BITS

FIFO

LFU

random

often pLRU approssimation

BIN tree

lines

A C B D

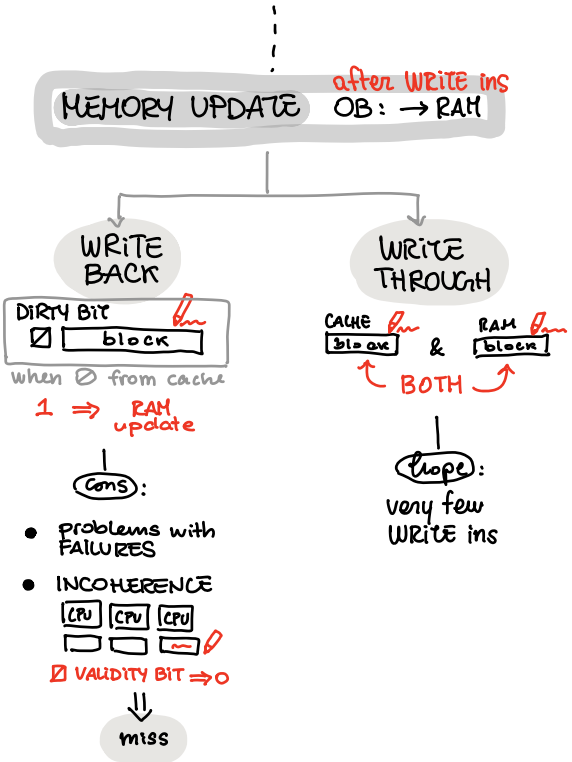
ORDER:

00 AE

10 BF

01 C

11 D


MEMORY UPDATE
after WRITE ins
OB: → RAM
WRITE BACK
WRITE THROUGH
DIRTY BIT
block
when ∅ from cache
1 ⇒ RAM update
CACHE
block
&
RAM
block
BOTH
Cons:
problems with FAILURES
INCOHERENCE
CPU
CPU
CPU
∅ VALIDITY BIT ⇒ 0
miss
Hope:
very few WRITE ins